# Smart Attendance System Project Guide

https://gemini.google.com/app/a6533d26e701acf4

*User prompt: How to do this*

Response: To successfully build the Smart Attendance Marking System for American Corner Mathara, you must first form a team of exactly three members. Every member must contribute to all phases of the project. You need to produce at least a Minimum Viable Product (MVP) to meet the 70% minimum passing score.

**1. Conceptualization (System Architecture)** Define the system architecture and draw your diagrams to outline how the hardware and software will communicate. Decide on your automated identification method, such as a QR code scanner or a biometric fingerprint module. You can easily pair these modules with a Wi-Fi-enabled microcontroller like an ESP32 to handle the data transmission. This diagram step accounts for 10% of your grade.

**2. Design (Business Logic)** Map out the exact data flow using flowcharts. This should visually represent the logic of the system: scanning an ID, verifying it, sending real-time data to the server, and logging the attendance to reduce administrative overhead. Developing this business logic and edge node concepts is worth another 10%.

**3. Simulation (Proof of Concept)** Before wiring physical components, validate your business logic using simulated sensor data. This serves as your Proof of Concept (POC) and completes the final 10% of the Technical Documentation & POC grading block (30% total).

**4. Platform Integration (ThingsBoard)** Configure your IoT platform using ThingsBoard. Start by setting up message generation nodes to simulate incoming attendance data. You will then build a dashboard to display accurate, real-time attendance data. The completed ThingsBoard Dashboard Development makes up 30% of your final score.

**5. Hardware Setup (Functional Prototype)** Transition your simulated setup into a physical, functional prototype. Wire your chosen scanner or biometric sensor to the microcontroller and ensure it successfully communicates with ThingsBoard. This physical hardware implementation is worth 30% of your grade.

To complete the project, prepare a Final Live Project Presentation, which accounts for the remaining 15% of the grading criteria.

Which identification method (QR or biometrics) are you leaning toward for the physical hardware?

---

*User prompt: QR*

Response: **Hardware Selection (ESP32 & Scanner)** An ESP32 is an excellent choice for the primary edge node, as its built-in Wi-Fi perfectly handles IoT platform communication. Pair it with a dedicated UART QR code scanner module (such as the GM65). Dedicated scanners handle the heavy image processing internally and simply send the decoded string (like a student ID number) to the ESP32 via serial (RX/TX) pins, making the system highly reliable for an American Corner Matara event environment.

**Business Logic & Code Flow**

1. **Initialize:** The ESP32 connects to the local Wi-Fi and connects to the ThingsBoard MQTT broker using its unique device access token.
2. **Listen:** The ESP32 continuously monitors its hardware serial port for incoming data from the QR scanner.
3. **Package:** When a QR code is scanned, the ESP32 reads the string and formats it into a JSON payload (e.g., {"student_id": "MTR-001", "status": "Present"}).
4. **Transmit:** The ESP32 publishes this JSON payload to the ThingsBoard telemetry topic (v1/devices/me/telemetry).

**ThingsBoard Integration** Create a new device in ThingsBoard and copy its MQTT access token to include in your ESP32 code. In your ThingsBoard Rule Chain, ensure there is a "Save Timeseries" node so the platform logs the exact timestamp of every incoming MQTT message.

**Dashboard Development** To satisfy the grading requirement for a real-time tracking interface, construct your ThingsBoard dashboard using an **Entities Table** widget. Configure the widget's data source to pull from your ESP32 device, setting the columns to display the student_id, status, and the automatically generated timestamp. You can also add a **Time Series Bar Chart** to visualize peak arrival times during classes.

**Simulation Step** Before buying or wiring the scanner, write a basic ESP32 sketch that generates a dummy student ID every 10 seconds and sends it to ThingsBoard. This fulfills the Proof of Concept (Simulation) grading requirement and confirms your network architecture is functioning correctly.

---

*User prompt: Draw a system architecture diagram like this.*

Response: Based on your request to adapt the system architecture diagram from image_2.png for your QR-based Smart Attendance Marking System, I have created a detailed block diagram using the same minimalist, black-background style.

This diagram adapts the three-section structure:

1. **Left Column (External Subject):** "Wachine machine" is replaced with **"Attendee (with QR ID)"**.
2. **Central Container (Edge Node):** The internal components are updated to include a **"QR Code Scanner"** and a **"Confirmation LED"**, wired to the **"ESP32"**. I have updated the data flow labels (e.g., "Scan Event Data (MQTT)", "Attendee ID / Timestamp").

3. **Right Column (ThingsBoard IOT Platform):** The list of analysis areas on the ThingsBoard side is updated to be relevant to your attendance system. I have included data points like **"Real-Time Count?"**, **"Peak Arrival Times?"**, and **"Duplicate Scan Alerts?"**, while maintaining the visual style of image_2.png.

---

*User prompt: Give me the component list*

Response: To transition from your simulated proof of concept to the physical hardware setup for the attendance system, you will need the following components.

**Core Processing & Connectivity**

- **ESP32 Development Board:** Acts as the central edge node, handling UART communication from the scanner and packaging MQTT payloads via Wi-Fi to ThingsBoard.
- **Micro-USB or USB-C Cable:** Required to flash your code to the ESP32 and provide power during the initial bench testing.

**Data Acquisition**

- **GM65 QR Code Scanner Module (or similar):** A dedicated hardware scanner that processes the QR image onboard and outputs a clean text string (the attendee ID) over serial (RX/TX) to the ESP32.

**Local Feedback**

- **5mm LEDs (Green and Red):** Wired to the ESP32 GPIO pins to provide immediate visual confirmation of a successful scan or a network transmission error.
- **Resistors (220Ω or 330Ω):** Essential current-limiting resistors to prevent the LEDs from drawing too much power and burning out.
- **Active Buzzer:** Provides a distinct audio "beep" for successful scans, which speeds up the physical queue during event registration.

**Prototyping & Power**

- **Solderless Breadboard:** For wiring and validating the circuit logic before designing a permanent printed circuit board for the final enclosure.
- **Jumper Wires (Male-to-Male and Male-to-Female):** For routing connections between the ESP32, the scanner module, and the feedback components.
- **5V USB Power Bank or Wall Adapter:** Provides independent power to run the system smoothly during the final live presentation without relying on a laptop.

---

*User prompt: For LEDs, I prefer one RGB LED for all indications; for power, a TP4056 with a LiPo battery. I want to etch a PCB and suggest enclosure dimensions. Update the component list. Don't need a buzzer. I want each and every component for this project, from screw to MCU.*

Response: Here is the comprehensive hardware manifest for an etched-PCB, battery-powered attendance terminal with an integrated RGB indicator.

**Core Processing & Data Acquisition**

- **ESP32-WROOM-32 Development Board:** The central edge node for Wi-Fi MQTT transmission.
- **GM65 QR Code Scanner Module:** Includes a 4-pin UART pigtail cable to interface directly with the ESP32.

**Power Management**

- **TP4056 Charging Module (with DW01A protection):** Manages LiPo charging via USB and prevents battery over-discharge.
- **18650 Lithium-Ion Battery (3.7V):** Provides reliable, high-capacity portable power.
- **18650 Single Battery Holder:** For easy battery mounting and replacement without spot-welding.
- **MT3608 DC-DC Step-Up Boost Converter:** Steps the 3.7V battery voltage up to a stable 5V for the ESP32's VIN pin.
- **SPDT Slide Switch:** Placed between the battery positive terminal and the boost converter to act as the main ON/OFF power switch.

**Visual Indication**

- **5mm RGB LED (Common Cathode):** A single component to display network status (e.g., blue), successful scan (green), or error (red).
- **3x 330Ω Resistors (Through-hole, 1/4 Watt):** Current-limiting resistors for the Red, Green, and Blue legs of the LED.

**PCB Etching & Mounting Components**

- **Single-Sided Copper Clad Board (FR4):** Blank canvas for your toner-transfer circuit traces.
- **Ferric Chloride (FeCl3):** Chemical etchant to remove unmasked copper.
- **Glossy Transfer Paper & Acetone:** For transferring your circuit design via heat and cleaning the board post-etch.
- **0.8mm and 1.0mm Drill Bits:** For drilling through-holes for components and headers.
- **Female Pin Headers (2.54mm pitch):** Cut to size to socket the ESP32, TP4056, and MT3608 so they can be removed from the etched board if needed.
- **22 AWG Solid Core Wire:** For patching any broken traces or making board-to-panel connections.

**Hardware & Enclosure Fasteners**

- **4x M3 Brass Standoffs (10mm) & 8x M3 Machine Screws (6mm):** To elevate and mount your custom etched PCB inside the housing.
- **4x M2 Machine Screws (4mm):** To securely mount the GM65 scanner module to the front faceplate.
- **Suggested Enclosure Dimensions:** 120mm (Length) x 85mm (Width) x 45mm (Height). This provides sufficient clearance for the 18650 battery holder, the socketed ESP32, the scanner module's focal depth, and panel-mounting the RGB LED and switch.

---

*User prompt: Where are the female headers for mounting the ESP32 to the PCB, and where are the connectors for connecting the QR code reader and battery?*

Response: **ESP32 Mounting Headers**

- **2x 19-pin Female Pin Headers (2.54mm pitch):** For socketing a standard 38-pin ESP32-WROOM-32 DevKitC. (If you are using the 30-pin version, you will need 2x 15-pin headers). It is most practical to purchase standard 40-pin breakaway female header strips and cut them to the exact size you need.

**QR Code Scanner Connector**

- **1x 4-pin JST-XH Connector (PCB Mount, 2.54mm pitch) OR 1x 4-pin Male Header:** The GM65 scanner includes a pigtail cable. Soldering a 4-pin JST-XH receptacle to your etched PCB provides a secure, clip-in connection for the scanner's RX, TX, 3.3V, and GND wires, preventing them from vibrating loose.

**Battery & Power Routing Connectors**

- **1x 2-pin Screw Terminal Block (KF301-2P or KF128-2P, 5.0mm pitch):** Soldered to your PCB to accept the raw positive and negative wires coming from the 18650 battery holder. This allows you to easily connect and disconnect the battery holder without desoldering.
- **Breakaway Male Headers (2.54mm pitch):** You will also want a few small sections of male pins to solder the TP4056 and MT3608 modules directly onto your main etched board, effectively creating a clean motherboard-and-daughterboard layout.

---

*User prompt: now give me the all component list*

Response: **Core Processing & Data Acquisition**

- **1x ESP32-WROOM-32 Development Board:** Central microcontroller handling UART communication from the scanner and MQTT telemetry over Wi-Fi.
- **1x GM65 QR/Barcode Scanner Module:** Hardware decoding scanner with integrated optics and UART interface.
- **1x 4-Pin JST-XH Pigtail Cable:** Included with the GM65 for serial communication.

**Power Subsystem**

- **1x 3.7V 18650 Lithium-Ion / LiPo Cell:** System power source.
- **1x Single 18650 Battery Holder with Leads:** Secure mounting for the cell.
- **1x TP4056 Lithium Battery Charger Module (with DW01A/FS8205A protection ICs):** Micro-USB/USB-C charging and over-discharge/over-current protection.
- **1x MT3608 DC-DC Step-Up Boost Converter Module:** Boosts 3.7V battery output to a regulated 5V rail for the ESP32 VIN and scanner.
- **1x SPDT Slide Switch (3-pin, panel mount or PCB mount):** Main ON/OFF power cut-off between battery and boost module.

**Visual Indication**

- **1x 5mm RGB LED (Common Cathode):** Multi-color visual status indicator for power, Wi-Fi connection, and scan results.
- **3x 330Ω Resistors (Through-Hole, 1/4W):** Current-limiting resistors for Red, Green, and Blue anodes.

**Connectors & Sockets**

- **2x 19-pin (or 2x 15-pin) 2.54mm Female Pin Headers:** Sockets for mounting the ESP32 to the PCB without direct soldering.
- **1x 40-pin 2.54mm Breakaway Male Pin Header Strip:** Solder pins for TP4056 and MT3608 daughterboard mounting.
- **1x 4-pin JST-XH 2.54mm PCB Header (Male):** Board receptacle for the GM65 scanner pigtail.
- **1x 2-Pin Screw Terminal Block (KF301-2P or KF128-2P, 5.0mm / 5.08mm pitch):** PCB connector for battery holder wire leads.

**PCB Fabrication & Soldering Materials**

- **1x Single-Sided Copper Clad FR4 Board (approx. 10cm x 15cm):** Substrate for circuit etching.
- **Ferric Chloride (*$FeCl_3$*) Etchant Solution:** Chemical agent for copper removal.
- **Glossy Photo Paper / Toner Transfer Sheets:** For laser-printer toner transfer.
- **Acetone & Steel Wool / Fine Sandpaper:** For copper cleaning and toner stripping.
- **0.8mm & 1.0mm Micro Drill Bits:** For drilling component through-holes.
- **Lead-Free / 60-40 Solder Wire & Rosin Flux:** For electrical connections.
- **22 AWG Solid-Core Hookup Wire:** For jumper links and panel-mount routing.

**Mechanical Fasteners & Enclosure**

- **4x M3 Brass Standoffs (10mm, Female-Female or Male-Female):** To elevate the etched PCB from the enclosure base.
- **8x M3 Machine Screws (6mm):** For securing the PCB to standoffs and the enclosure base.
- **4x M3 Hex Nuts:** For standoff anchoring.
- **4x M2 Machine Screws (4mm–6mm):** Front-bezel mounting screws for the GM65 scanner.
- **Custom Enclosure (Recommended Dimensions: 120mm × 85mm × 45mm):** Housing sized for the internal battery holder, stacked PCB modules, and scanner clearance.

---

*User prompt: This is my supervisor message: Good thinking on the project and the overall approach. However, the current component list seems to be taking the solution in a slightly more complex direction. I'd recommend considering the solution from the opposite side. We can use an ESP32 with a built-in display to generate and display a *dynamic QR code, which can be controlled by the IoT platform. Users can simply scan the QR code and mark their attendance through the link provided. For better security, instead of displaying a single static QR code, we can implement *multiple rotating QR codes carrying dynamic data. This can help prevent reuse of a previously captured QR code. You can refer to this resource for a practical implementation approach:*
*https://blog.adafruit.com/2026/05/27/move-data-between-two-devices-via-qr-codes-and-an-html-page/*

Response: Flipping the logic so the terminal displays a rotating QR code for attendees to scan with their own smartphones drastically reduces your hardware footprint. This approach shifts the heavy lifting from physical components to software and leverages the screens everyone already carries.

**Dynamic QR Code Workflow**

- **Token Generation:** ThingsBoard (or your backend server) generates a secure, time-stamped cryptographic token every 15–30 seconds and publishes it to the ESP32 via MQTT.

- **Display Rendering:** The ESP32 receives the MQTT payload, generates a QR code containing a web link (e.g., [portal.com/checkin?token=XYZ](https://portal.com/checkin?token=XYZ)), and renders the matrix on its built-in screen.
- **User Action:** Attendees arriving at American Corner Matara scan the screen with their phone cameras, which opens a simple HTML form where they submit their ID.
- **Anti-Spoofing Security:** Because the token rotates continuously, a photograph of the QR code sent to an absent friend will expire before it can be used, ensuring only physically present students can register.

**Updated Hardware Manifest**

| Component | Role in Revised Architecture | Status |
|---|---|---|
| **ESP32 with Integrated TFT (e.g., LilyGO TTGO T-Display)** | Replaces the standard dev board. Handles Wi-Fi, receives MQTT payloads, and draws the dynamic QR matrix directly on its screen. | **Add** |
| **GM65 QR Code Scanner & Pigtail** | No longer needed. The scanning hardware is now the attendee's personal smartphone. | **Remove** |
| **RGB Status LED & Resistors** | Redundant. The built-in TFT screen can easily display color-coded status banners for network connectivity or low battery. | **Remove** |
| **TP4056, MT3608, 18650 Battery System** | Still essential for providing portable, cordless power to the terminal during events. | **Keep** |
| **Custom Etched PCB** | Greatly simplified. Now only serves as a power distribution board to link the battery modules to the ESP32 display board. | **Modify** |

This pivot simplifies your physical fabrication since you no longer need to align and mount a bulky optical scanner inside the enclosure. Your main technical challenge will now shift to the C++ side, utilizing a library like TFT_eSPI alongside a QR generator to continuously draw and refresh the incoming MQTT strings into a readable visual matrix on the screen.

---

*User prompt: Can you read this page: https://blog.adafruit.com/2026/05/27/move-data-between-two-devices-via-qr-codes-and-an-html-page/*

Response: The article highlights a project called **ShadowCat**, which is a single-file, offline HTML page designed to transfer data between two devices using nothing but screens and cameras.

Here are the key takeaways from the page and how they directly apply to your supervisor's feedback:

**How the Concept Works (ShadowCat):**

- **Data to QR Conversion:** The software takes data (even whole files) and encodes it into QR codes.
- **Rapid Rotation (FPS):** For larger data, it splits the information into "chunks" and generates a rapid sequence of QR codes. It then cycles through these codes at a set frame rate (FPS), essentially turning the QR codes into a video.
- **Optical Reception:** The receiving device points its camera at the screen, reads the rapid sequence of QR codes, pieces the chunks back together, and verifies the data.
- **Browser Permissions:** The article notes that for a web page to access a phone's camera to scan these codes, the page must be served over a secure connection (HTTPS) or via localhost, otherwise the browser will block the camera.

**Applying this to your Attendance System:** Your supervisor is suggesting you use a simplified version of this "rotating QR" concept to drastically increase security.

Instead of breaking a large file into chunks like ShadowCat does, your ESP32 display will rotate different, self-expiring URLs or tokens.

1. **Dynamic Generation:** Your ThingsBoard platform generates a new, unique token every few seconds.
2. **ESP32 Display:** The ESP32 receives the token via MQTT, generates a new QR code matrix (e.g., [yourdomain.com/attend?token=1A2B3C](https://yourdomain.com/attend?token=1A2B3C)), and flashes it on the TFT screen.
3. **The Anti-Cheat Mechanism:** Because the QR code on the ESP32 screen changes every few seconds, a student cannot simply take a photo of the screen and WhatsApp it to a friend who is skipping the event. By the time the friend scans the photo, that specific token will have already expired on your backend.

This approach eliminates the need for you to wire up a physical GM65 scanner module. Instead, you utilize the cameras already built into the attendees' smartphones, routing them to a secure HTML form to log their presence.

---

*User prompt: Let's say one student came and scan the qr code and mark his attendence. Can the student scan another QR code and mark another student's attendance?*

Response: Yes, they absolutely could. While the rotating QR code prevents **remote cheating** (taking a photo of the screen and sending it to a friend at home), it does not natively prevent **local buddy-punching** (one person standing in the room rapidly scanning the screen to enter multiple friends' IDs).

Since your hardware is now acting only as a display, you must solve this on the software side within your HTML form and backend logic.

Here are the most effective ways to prevent one device from submitting multiple attendances:

- **Browser Cookies & Local Storage (Low Friction):** Once a student submits the HTML form successfully, your web page drops a "checked-in" cookie or local storage flag on their mobile browser. If they scan the next rotating QR code and load the page again, the site reads the cookie and displays a message like "Attendance already recorded for this device," hiding the submission form.
    - *Note:* A tech-savvy student can bypass this by using Incognito Mode or clearing their cache between scans.
- **Authenticated Login (High Security):** Instead of an open form where anyone can type an ID string, the QR code URL redirects them to a secure login page. The student must authenticate (e.g., "Sign in with Google" using their registered email). The backend ensures that one specific user account can only log a single attendance record per session, making it impossible to submit an ID for someone else unless they have their friend's email password.
- **Device Fingerprinting & Rate Limiting:** Your backend can run a script to collect non-personal device data (OS, browser version, screen resolution) to create a unique "fingerprint" of the phone. If your ThingsBoard or web server detects multiple distinct student IDs being submitted from the exact same digital fingerprint within a 2-minute window, it automatically flags or rejects the subsequent attempts.

Since this system is for American Corner Matara events, do the attendees already use registered email addresses or student portal accounts that you could hook into for an authenticated login?

---

*User prompt: authenticated login better*

Response: Integrating authenticated login forces the system to verify *who* is scanning the code, completely neutralizing local buddy-punching. Because the user's identity is securely tied to their login credentials, they physically cannot submit an attendance record for someone else without having their friend's actual password.

**The Authenticated Data Flow**

- **The Scan:** The ESP32 screen displays a QR code containing a dynamic URL (e.g., acm-portal.web.app/checkin?token=8F2A9). The student scans it.
- **The Identity Gate:** The browser opens the URL but hides the check-in button behind a mandatory login screen (such as a "Sign in with Google" prompt). This ensures the identity being submitted is the verified owner of the account.
- **The Dual-Payload:** Once the student successfully logs in, your web application silently combines their verified email address with the time-sensitive token pulled from the URL. It sends both variables as a JSON payload to your ThingsBoard instance.
- **The Rule Chain Verification:** Inside ThingsBoard, your Rule Chain intercepts the incoming payload and verifies two conditions:
    1. Is the token still valid and unexpired?
    2. Has this specific email address already logged a "Present" status for today's session?
- **The Lockout:** If the student tries to scan the next rotating QR code a few seconds later to register a friend, the web app will recognize their active session. ThingsBoard will instantly reject the second

attempt because their email is already flagged as present for the day.

To build this efficiently without spinning up a heavy server, you can host your HTML and JavaScript on a free platform like Firebase Hosting. Firebase Authentication allows you to drop in a pre-built Google login button with minimal code, handling the secure session state directly on the student's phone.

Are you planning to build the web portal yourself, or is one of your two project team members handling the web and ThingsBoard integration?

---

*User prompt: No, I'm gonna build the whole project.*

Response: Taking on the complete build means orchestrating the embedded firmware, the web authentication, and the IoT platform routing simultaneously. Breaking the architecture into three distinct layers keeps the development manageable.

- **Embedded Firmware (C++):** Utilize TFT_eSPI to manage the screen rendering alongside your standard Wi-Fi and MQTT libraries. The ESP32 subscribes to a specific ThingsBoard topic to receive the rotating cryptographic tokens. Upon receiving a new payload, the code appends the token to your web portal's base URL and uses an ESP-compatible library (like ESPQRcode) to draw the updated matrix directly on the display.
- **Authentication Frontend (HTML/JS):** Host a single-page application on a free platform like Firebase Hosting. When an attendee scans the screen, they land on this page, which extracts the unique token from the URL and prompts a Google Sign-In via Firebase Auth. This establishes the attendee's verified identity.
- **Data Routing & Validation (Python):** Instead of exposing your ThingsBoard credentials directly on the public frontend, route the web submission to a lightweight Python backend using Flask or FastAPI. This script verifies the token's timestamp, confirms the user's authentication state, and acts as a secure bridge to publish the clean JSON payload to ThingsBoard.
- **ThingsBoard Visualization:** Configure an Entities Table on your dashboard to display the final telemetry data. You can implement a Rule Chain script here to filter out duplicate check-ins if the same email address appears twice on the same day.

Which layer do you want to tackle first: configuring the C++ display logic, or setting up the web authentication portal?

---

*User prompt: configuring the C++ display logic*

Response: To render the rotating QR codes on your ESP32 display, you will leverage your existing experience with the TFT_eSPI library alongside a lightweight QR generation library. While tackling this firmware layer, remember the project grading rubric strictly mandates that teams consist of exactly three members, and every member is expected to contribute to all project phases. You will need to ensure your two teammates are actively collaborating on this code design, or handling its corresponding simulation and documentation, to prevent any point deductions.

**Core Library Setup** Install the qrcode library (by ricmoo) from the Arduino Library Manager. You will also need to ensure your User_Setup.h file within the TFT_eSPI folder is configured properly for your specific

display driver and hardware pins.

**The Rendering Logic** When your MQTT callback function receives a new token string from ThingsBoard, the ESP32 must combine it with your web portal's base URL. The qrcode library then translates this URL string into a 2D grid of booleans (representing black and white squares). Finally, you loop through this grid using tft.fillRect() to scale and draw the geometric modules onto the screen.

**Example Firmware Implementation**

```
#include <TFT_eSPI.h>
#include <qrcode.h>

TFT_eSPI tft = TFT_eSPI();
QRCode qrcode;

void displayDynamicQR(String token) {
    // Construct the secure URL
    String full_url = "https://acm-portal.web.app/checkin?token=" + token;

    // Clear the screen with a white background for high contrast
    tft.fillScreen(TFT_WHITE);

    // Version 3 QR code supports up to 55 alphanumeric characters
    uint8_t qrcodeData[qrcode_getBufferSize(3)];
    qrcode_initText(&qrcode, qrcodeData, 3, 0, full_url.c_str());

    // Calculate dynamic scaling to center the QR code on the TFT screen
    int scale = min(tft.width() / qrcode.size, tft.height() / qrcode.size);
    int offsetX = (tft.width() - (qrcode.size * scale)) / 2;
    int offsetY = (tft.height() - (qrcode.size * scale)) / 2;

    // Render the matrix
    for (uint8_t y = 0; y < qrcode.size; y++) {
        for (uint8_t x = 0; x < qrcode.size; x++) {
            if (qrcode_getModule(&qrcode, x, y)) {
                tft.fillRect(offsetX + (x * scale), offsetY + (y * scale),
scale, scale, TFT_BLACK);
            }
        }
    }
}
```

What specific TFT display driver (e.g., ST7789, ILI9341) is integrated into your ESP32 board for this terminal?

---

*User prompt: ILI9341*

Response: Configuring the TFT_eSPI library for the ILI9341 driver requires editing its central configuration

file so the ESP32 routes the SPI signals correctly.

### 1 Locate User_Setup.h

File Directory

Open your Arduino environment's library folder, navigate to TFT_eSPI, and open the User_Setup.h file in a text editor.

### 2 Enable the ILI9341 Driver

Code Edit

Search for the line //#define ILI9341_DRIVER and remove the forward slashes (//) to uncomment it. Ensure all other driver definitions (like ST7789 or ILI9341_2) remain commented out.

### 3 Assign ESP32 SPI Pins

Hardware Mapping

Scroll down to the ESP32 hardware section and map your specific wiring. The ILI9341 uses 3.3V logic, which pairs perfectly with the ESP32 without logic level shifters. A standard, reliable SPI pin mapping is:

```
#define TFT_MISO 19
#define TFT_MOSI 23
#define TFT_SCLK 18
#define TFT_CS   15  // Chip select control pin
#define TFT_DC    2  // Data Command control pin
#define TFT_RST   4  // Reset pin
```

### 4 Verify Display Functionality

Compilation

Upload the previous C++ QR rendering sketch to the ESP32. If the configuration is successful, the ILI9341 will instantly clear to a white background and draw a crisp, scaled QR code matrix in the center of the screen.

---

*User prompt: Give me the full esp32 code*

Response: Here is the complete C++ firmware integrating the Wi-Fi connection, ThingsBoard MQTT subscription, and ILI9341 display rendering.

Before compiling, ensure you have installed **ArduinoJson** alongside TFT_eSPI, qrcode, and PubSubClient via the Arduino Library Manager to securely parse the incoming payload.

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <TFT_eSPI.h>
#include <qrcode.h>
#include <ArduinoJson.h>

// --- Network & ThingsBoard Configuration ---
const char* WIFI_SSID     = "YOUR_WIFI_SSID";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
const char* TB_SERVER     = "demo.thingsboard.io"; // Change if using a local
server
const int   TB_PORT       = 1883;
const char* TB_TOKEN      = "YOUR_ESP32_ACCESS_TOKEN";

// --- Global Objects ---
WiFiClient espClient;
PubSubClient client(espClient);
TFT_eSPI tft = TFT_eSPI();
QRCode qrcode;

// --- Web Portal Base URL ---
const String BASE_URL = "https://acm-portal.web.app/checkin?token=";

void setup_wifi() {
  tft.fillScreen(TFT_BLACK);
  tft.setCursor(10, 10);
  tft.setTextColor(TFT_WHITE);
  tft.setTextSize(2);
  tft.println("Connecting WiFi...");

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    tft.print(".");
  }
  tft.println("\nWiFi Connected!");
}

void displayDynamicQR(String token) {
  String full_url = BASE_URL + token;
  tft.fillScreen(TFT_WHITE);
```

```
  // Version 3 QR code supports up to 55 alphanumeric characters
  uint8_t qrcodeData[qrcode_getBufferSize(3)];
  qrcode_initText(&qrcode, qrcodeData, 3, 0, full_url.c_str());

  // Calculate dynamic scaling and centering
  int scale = min(tft.width() / qrcode.size, tft.height() / qrcode.size);
  int offsetX = (tft.width() - (qrcode.size * scale)) / 2;
  int offsetY = (tft.height() - (qrcode.size * scale)) / 2;

  // Render the matrix
  for (uint8_t y = 0; y < qrcode.size; y++) {
    for (uint8_t x = 0; x < qrcode.size; x++) {
      if (qrcode_getModule(&qrcode, x, y)) {
        tft.fillRect(offsetX + (x * scale), offsetY + (y * scale), scale,
scale, TFT_BLACK);
      }
    }
  }
}

void callback(char* topic, byte* payload, unsigned int length) {
  // Convert payload to String
  String message = "";
  for (int i = 0; i < length; i++) {
    message += (char)payload[i];
  }

  // Parse ThingsBoard JSON (assumes payload is
{"token":"YOUR_DYNAMIC_STRING"})
  StaticJsonDocument<200> doc;
  DeserializationError error = deserializeJson(doc, message);

  if (!error) {
    if (doc.containsKey("token")) {
      String newToken = doc["token"].as<String>();
      displayDynamicQR(newToken);
    }
  }
}

void reconnect() {
  while (!client.connected()) {
    if (client.connect("ESP32_Attendance_Node", TB_TOKEN, NULL)) {
      // Subscribe to ThingsBoard Shared Attributes to receive rotating tokens
      client.subscribe("v1/devices/me/attributes");
    } else {
      delay(5000);
    }
  }
}

void setup() {
```

```
  Serial.begin(115200);

  // Initialize Display
  tft.init();
  tft.setRotation(1); // Landscape mode

  setup_wifi();

  // Initialize MQTT
  client.setServer(TB_SERVER, TB_PORT);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();
}
```

**Key Code Mechanics**

- **Attributes Subscription:** The reconnect() function subscribes to v1/devices/me/attributes. When you update the "token" shared attribute for this device in ThingsBoard, it instantly pushes the new value to the ESP32.
- **JSON Parsing:** The MQTT callback receives a raw byte array. ArduinoJson securely decodes this structure to extract just the token value before passing it to the display function.
- **Landscape Rendering:** tft.setRotation(1) ensures the code maximizes the screen real estate of your ILI9341 display.

---

*User prompt: let's setting up the web authentication portal*

Response: **Firebase Console Configuration**

1. Create a new Firebase project and register a Web App to generate your unique firebaseConfig object.
2. Navigate to Authentication > Sign-in method and enable the **Google** provider, ensuring you also configure your support email address.
3. Under the Settings > Authorized domains tab, verify that your hosting domain (e.g., acm-portal.web.app or localhost for local testing) is listed so Firebase permits the OAuth redirect sequence.

**Web Portal Code (index.html)** Create a single HTML file. This structure utilizes the modular Firebase SDK via CDN to trigger a Google authentication pop-up, extract the rotating token generated by the ESP32 from the URL parameters, and package the final JSON payload.

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>American Corner Matara - Check In</title>
    <style>
        body { font-family: Arial, sans-serif; text-align: center; padding:
50px; }
        #login-btn { padding: 10px 20px; font-size: 16px; cursor: pointer; }
    </style>
</head>
<body>
    <h2>Attendance Check-In</h2>
    <p id="status">Please log in to mark your attendance.</p>
    <button id="login-btn">Sign in with Google</button>

    <script type="module">
        import { initializeApp } from
"https://www.gstatic.com/firebasejs/10.7.1/firebase-app.js";
        import { getAuth, signInWithPopup, GoogleAuthProvider } from
"https://www.gstatic.com/firebasejs/10.7.1/firebase-auth.js";

        // Replace with your Firebase project config variables
        const firebaseConfig = {
            apiKey: "YOUR_API_KEY",
            authDomain: "YOUR_AUTH_DOMAIN",
            projectId: "YOUR_PROJECT_ID",
            appId: "YOUR_APP_ID"
        };

        const app = initializeApp(firebaseConfig);
        const auth = getAuth(app);
        const provider = new GoogleAuthProvider();

        document.getElementById('login-btn').addEventListener('click', () => {
            signInWithPopup(auth, provider)
                .then((result) => {
                    const user = result.user;

                    // Extract the rotating token from the URL (e.g.,
?token=XYZ)
                    const urlParams = new
URLSearchParams(window.location.search);
                    const token = urlParams.get('token');

                    if (!token) {
                        document.getElementById('status').innerText = "Error:
Invalid or missing QR token.";
                        return;
                    }

                    // Prepare the final attendance payload
                    const attendanceData = {
                        student_email: user.email,
                        student_name: user.displayName,
                        qr_token: token
```

```
                    };

                    document.getElementById('status').innerText = `Welcome,
${user.displayName}! Attendance logged.`;
                    document.getElementById('login-btn').style.display =
'none';

                    // TODO: Send 'attendanceData' to your data routing layer
here
                    console.log("Payload ready for routing:", attendanceData);
                })
                .catch((error) => {
                    console.error("Authentication Error:", error.message);
                });
        });
    </script>
</body>
</html>
```

**Data Routing Options** Once the frontend isolates the student_email and qr_token, you must route this payload to your IoT architecture for Rule Chain validation.

- **Direct ThingsBoard API (Less Secure):** Use the JavaScript fetch() API directly within the .then() block to POST the JSON payload to a ThingsBoard device telemetry endpoint. This requires exposing a device access token in your client-side code.
- **Python Webhook Bridge (Highly Secure):** Set up a lightweight Python backend using Flask or FastAPI. The frontend fetch() API sends the payload to this Python bridge. The script acts as a secure gatekeeper, validating the payload's integrity before securely publishing it to ThingsBoard via an MQTT client, hiding all platform credentials from the public web page.

What method will you use to route the final JSON payload from this web frontend into the ThingsBoard platform?

---

*User prompt: where to host the Web Portal*

Response: For a single-page HTML application utilizing Firebase Authentication, the hosting platform must enforce HTTPS to allow secure OAuth redirects.

**Firebase Hosting (Recommended)**

- Because you must create a Firebase project to enable the Google Sign-In, deploying your HTML file directly through the Firebase CLI keeps your entire web layer in one ecosystem.
- It automatically provisions a free SSL certificate and provides a clean subdomain (e.g., acm-attendance.web.app).

**GitHub Pages**

- This is highly effective if you are already using GitHub to manage version control and share the project code with your two project team members.
- Uploading your index.html to a public repository and enabling the Pages feature in the settings provides instant, free hosting on a secure github.io domain.

**Netlify or Vercel**

- Both platforms allow you to bypass command-line tools entirely by dragging and dropping your project folder directly into their web interface for immediate deployment.
- They provide free tiers, auto-renewing SSL certificates, and fast global content delivery so the portal loads quickly on attendees' smartphones.

If you plan to implement the secure Python routing bridge discussed earlier instead of a direct frontend API call, you will also need a separate backend host (like Render or PythonAnywhere) to run that routing script continuously.

---

*User prompt: Let's use Firebase*

Response: Deploying your portal through Firebase Hosting keeps your frontend code and authentication rules securely managed within the same ecosystem.

**1. Install the Firebase CLI**

- Open your computer's terminal or command prompt.
- Run the command npm install -g firebase-tools. (Note: This requires having Node.js installed on your computer).

**2. Authenticate your Local Machine**

- Run firebase login in the terminal.
- This will open a browser window asking you to log in with the same Google account you used to create the Firebase project.

**3. Initialize the Project Structure**

- Create a new folder on your computer for this web project (e.g., attendance-portal).
- Inside that folder, create a subfolder named public.
- Move your index.html file into this public folder.
- Navigate to the attendance-portal folder in your terminal and run firebase init hosting.

**4. Configure the Hosting Settings**

- Use your arrow keys to select **"Use an existing project"** and hit Enter. Choose the Firebase project you set up earlier.
- When asked "What do you want to use as your public directory?", type public and press Enter.
- When asked "Configure as a single-page app (rewrite all urls to /index.html)?", type y (Yes). This is critical so the ?token= URL parameters don't cause 404 errors.
- When asked "Set up automatic builds and deploys with GitHub?", type N (No).

- When asked to overwrite index.html, type N (No) to protect the code you already wrote.

**5. Deploy to the Web**

- Run the command firebase deploy.
- Firebase will upload your files and return a secure **Hosting URL** (e.g., [https://your-project-id.web.app](https://your-project-id.web.app)).

Take this final Hosting URL and paste it into the BASE_URL variable in your ESP32 C++ firmware. Every time your ESP32 generates a QR code, it will now point attendees directly to this live, secure web portal to authenticate.

---

*User prompt: I got the hosting URL. I'm also responsible for ThingsBoard integration.*

Response: With the frontend hosted, ThingsBoard will now serve two distinct roles: generating the rotating tokens for the ESP32 to display, and receiving the final attendance data from your Firebase portal.

Here is how to configure the platform, keeping the specific grading requirement to use "message generation nodes" in mind.

**1. Device Provisioning**

- Navigate to **Entities > Devices** in ThingsBoard and add a new device (e.g., "Attendance Terminal 1").
- Open the device details, click **Manage Credentials**, and copy the **Access Token**. You will need to paste this into your ESP32 C++ code, and it will also be used by your web portal to send data.

**2. Token Generation (Rule Chain)** To satisfy the project requirement of using message generation nodes, we will build a loop that creates the dynamic QR tokens.

- Go to **Rule Chains** and open your Root Rule Chain.
- Drag a **Generator** node onto the canvas. Set its generator cycle to 15 seconds.
- Drag a **Transformation > Script** node. Connect the Generator to this script, and write a simple JavaScript function to generate a random alphanumeric string:

```
var randomToken = Math.random().toString(36).substring(2, 10);
return {msg: {token: randomToken}, metadata: metadata, msgType:
"POST_ATTRIBUTES_REQUEST"};
```

- Drag an **Action > Save Attributes** node and connect the Script node to it. Configure it to save as **Shared Attributes**.
- *How it works:* Every 15 seconds, this chain generates a new token and saves it as a shared attribute on your device. Because your ESP32 code is subscribed to shared attributes via MQTT, ThingsBoard will automatically push the new token to the display.

**3. Web Portal Data Ingestion** Now, you must update the TODO section in your Firebase HTML file to push the authenticated student data back to ThingsBoard.

Replace the console.log and TODO lines in your index.html with a direct HTTP POST request using the Fetch

API:

```
// Replace TB_URL and ACCESS_TOKEN with your actual ThingsBoard details
const tbUrl = 'https://demo.thingsboard.io/api/v1/YOUR_ACCESS_TOKEN/telemetry';

fetch(tbUrl, {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify(attendanceData)
})
.then(response => {
    if(response.ok) {
        document.getElementById('status').innerText = `Welcome,
${user.displayName}! Attendance securely logged.`;
    } else {
        document.getElementById('status').innerText = "Error syncing with
platform.";
    }
})
.catch(error => console.error('Error:', error));
```

*After adding this, run firebase deploy again in your terminal to update the live site.*

**4. Dashboard Development** Now that telemetry is flowing into the platform from the web portal, you can build the interface that accounts for 30% of your grade.

- Navigate to **Dashboards** and create a new one.
- Create an alias pointing to your "Attendance Terminal 1" device.
- Add an **Entities Table** widget. Configure the columns to display the student_name and student_email telemetry keys, along with the timestamp.
- Add a **Time Series Bar Chart** widget. Bind it to the incoming telemetry to visualize peak arrival times during the events.

Once your ESP32 is powered on, it will begin receiving the tokens and drawing the matrices. Have you successfully flashed the C++ code to the ESP32 to test if the screen updates every 15 seconds?

---

*User prompt: Explain, 4. Dashboard Development*

Response: The dashboard is the visual interface where event administrators will monitor the incoming data. Completing this step effectively secures the 30% grading weight allocated for ThingsBoard Dashboard Development.

Here is the step-by-step breakdown to build a professional interface using the telemetry data your Firebase portal is now sending.

**Step 1: Create the Entity Alias** Before adding widgets, you must tell the dashboard where to pull its data from.

1. Navigate to **Dashboards** and click the "+" icon to create a new dashboard.
2. Open the dashboard and click the **Edit** (pencil) icon in the bottom right corner.
3. Click the **Entity Aliases** icon at the top.
4. Add an alias named "Terminal" and set the filter type to **Single Entity**.
5. Select **Device** as the type and choose the specific device you created earlier (e.g., "Attendance Terminal 1"). Save the alias.

**Step 2: Build the Live Attendance Log** This widget will act as the digital sign-in sheet, fulfilling the project goal of providing accurate, real-time attendance data.

1. Click **Add new widget** and select the **Cards** bundle, or search for **Entities Table**.
2. Set the data source to the "Terminal" alias you just created.
3. In the Data Keys section, manually type in the exact keys your web portal is sending: student_name and student_email. Add them as telemetry keys.
4. ThingsBoard automatically includes a timestamp column. You can rename the column headers in the widget settings to be cleaner (e.g., "Name", "Email Address", "Scan Time").

**Step 3: Visualize Peak Arrival Times** Adding charts elevates the dashboard from a simple spreadsheet to an analytical tool, which is excellent for a final presentation.

1. Add another widget and select the **Charts** bundle. Choose the **Time Series Bar Chart**.
2. Select your "Terminal" alias as the data source.
3. Add a data key. Since you want to count total check-ins over time, you can map it to any telemetry key (like student_name) but change the aggregation function in the widget settings from "None" to **Count**.
4. Set the grouping interval to 5 or 10 minutes. This will generate a bar chart showing exactly when the highest volume of attendees arrived during the registration window.

**Step 4: Total Attendee Counter** A quick-glance total is highly useful for event organizers to know when a room is at capacity.

1. Add a **Digital Gauge** or a **Simple Card** widget.
2. Connect it to the "Terminal" alias.
3. Use the same Count aggregation method on one of your telemetry keys to display the total number of unique scans for the current session.

Once you have arranged these three widgets on the grid, click the orange checkmark in the bottom right to save the dashboard layout. Your platform is now fully configured to receive, log, and visualize the dynamic QR scans in real-time.

---